

HW4 – Genetic Programming for Maze Navigation

Mehdi Mortazavian

January 2026

1 Introduction

Genetic Programming (GP) is an evolutionary computation technique in which computer programs are automatically evolved to solve a given task. Unlike traditional genetic algorithms that operate on fixed-length vectors, GP represents solutions as executable program structures, typically in the form of trees. This representation allows GP to discover both the structure and the parameters of a solution simultaneously.

In this homework, Genetic Programming is applied to a classical control and navigation problem: guiding an agent through a maze from a fixed start location to a predefined goal. The agent must navigate through obstacles (walls) while minimizing inefficient behaviors such as hitting walls, revisiting the same locations, or taking unnecessarily long paths.

The primary objective of this assignment is to design and implement a complete GP system that satisfies the following requirements:

- Tree-based representation of programs
- Well-defined function and terminal sets
- Fitness evaluation based on navigation performance
- Use of evolutionary operators such as selection, crossover, and mutation
- Visualization of population behavior and evolutionary progress

The final implementation closely follows the provided homework documentation and reference notebook, while being written as a standalone Python program.

2 Problem Definition

The problem environment is defined as a two-dimensional grid maze. Each cell in the grid can either be a free space or a wall. The agent starts at the top-left corner of the maze and must reach the bottom-right corner.

2.1 Maze Representation

The maze is represented as a matrix where:

- Value 0 indicates a free cell
- Value 1 indicates a wall

Movement outside the grid boundaries is treated as a wall collision. This simplifies environmental constraints and ensures that all illegal movements are penalized.

2.2 Agent Capabilities

The agent can perform one of four primitive actions at each step:

- UP
- DOWN
- LEFT
- RIGHT

Each action attempts to move the agent by one grid cell in the corresponding direction. If the target cell is a wall or lies outside the maze, the agent remains in its current position and a wall hit is recorded.

3 Genetic Programming Representation

3.1 Tree-Based Programs

Each individual in the population is represented as a tree structure. Trees are encoded using nested Python lists and strings:

- Terminal nodes are movement commands (strings)
- Internal nodes are function symbols with child subtrees

This representation allows programs of variable size and depth, which is one of the key advantages of Genetic Programming.

3.2 Terminal Set

The terminal set consists of the primitive actions available to the agent:

$$T = \{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}$$

Terminal nodes directly interact with the environment and correspond to atomic behaviors.

3.3 Function Set

The function set contains a single sequencing operator:

$$F = \{\text{PROGN2}\}$$

The `PROGN2` function takes two child subtrees and executes them sequentially. This operator allows the construction of complex programs by composing simpler behaviors.

4 Program Execution Semantics

Program execution is performed recursively starting from the root of the tree. When a terminal node is encountered, the corresponding movement action is executed in the environment. When a function node is encountered, its children are evaluated in order.

Execution is terminated when either:

- The maximum allowed number of steps is reached
- The agent reaches the goal position

During execution, the following statistics are recorded:

- Total number of steps taken
- Number of wall hits
- Complete path of visited positions

These statistics form the basis of the fitness evaluation.

5 Fitness Function

The fitness function is designed as a minimization objective and is defined as:

$$F = s + 2d + 10w + 5l$$

where:

- s is the number of steps taken
- d is the Manhattan distance from the final position to the goal
- w is the number of wall hits
- l is the number of revisited cells (loops)

This formulation encourages short, direct, and collision-free paths. Wall hits and loops are heavily penalized to discourage inefficient behaviors.

A special case is defined for perfect solutions: if the agent reaches the goal without any wall hits, the fitness value is set to zero.

6 Population Initialization

The initial population is generated using the *Ramped Half-and-Half* method. This approach ensures diversity by creating individuals with varying tree depths and shapes.

Two initialization strategies are used:

- **Full:** All nodes are functions until the maximum depth is reached
- **Grow:** Nodes can be either functions or terminals before reaching the maximum depth

By alternating between these strategies, the population contains both deep and shallow programs with diverse structures.

7 Selection Mechanism

Fitness-proportionate (roulette wheel) selection is employed. Since the problem is a minimization task, fitness values are inverted so that individuals with lower fitness have a higher probability of being selected.

This selection method maintains diversity while still favoring better-performing individuals.

8 Genetic Operators

8.1 Subtree Crossover

Crossover is implemented as subtree exchange. A random subtree from one parent is selected and inserted into a randomly chosen location in the other parent. Depth constraints are enforced to prevent uncontrolled growth (bloat).

8.2 Subtree Mutation

Mutation is implemented by replacing a randomly selected subtree with a newly generated random subtree. This operator introduces novel genetic material into the population and helps avoid premature convergence.

9 Evolutionary Strategy

A steady-state evolutionary strategy is used. In each generation:

1. Two parents are selected
2. Two offspring are generated using crossover and mutation
3. Offspring are evaluated
4. The worst individuals in the population are replaced if offspring are better

This approach allows gradual improvement of the population while preserving high-quality solutions.

10 Visualization and Analysis

Several visualization tools are implemented to analyze GP behavior:

- Visualization of sample individuals from the first generation
- Plotting average and best fitness over generations
- Visualization of the best evolved path on the maze

These visualizations provide insight into both individual behaviors and overall evolutionary dynamics.

11 Conclusion

This homework demonstrates the effectiveness of Genetic Programming for solving navigation and control problems. By evolving executable programs rather than fixed parameters, GP is able to discover complex behaviors from simple primitives.

The implemented system satisfies all requirements of the assignment and closely follows the provided documentation. Experimental results show that GP can successfully evolve programs that guide the agent toward the goal while minimizing inefficient behaviors.

Future extensions could include richer function sets, memory mechanisms, or more complex sensor models.